neither of these files has changed, Docker will use the cached image from the previous build. If either of these files has changed, Docker will rebuild the image from this point on.

The next instruction will run `bundle install` to install the gems needed by the application. This step may take a while, which is why it's important to only be run if the `Gemfile` or `Gemfile.lock` file havs changed. It will also run bootsnap precompile on the Gemfile to speed up the start time of the application.[90]

Once this is complete, the rest of your application is copied to the image. Then, bootsnap is run again, this time on the application code and libraries.

The final step is to run `rails assets:precompile` to precompile the assets for the application. This is a step that's only needed when deploying the application. In development, the assets are served dynamically from the `app/assets` directory. The setting of `SECRET_KEY_BASE` is a work-around to Rails requiring a master key even on commands that won't make use of it.

Deploy Image

The last image in any Dockerfile is the image that will be run when the container is started. Note that this step too specifies the original image is to be used as the base. This avoids the inclusion of all of the build tools in the final image.

rails80/depot_td/Dockerfile

```
# Final stage for app image
FROM base

# Copy built artifacts: gems, application
COPY --from=build "${BUNDLE_PATH}" "${BUNDLE_PATH}"
COPY --from=build /rails /rails

# Run and own only the runtime files as a non-root user for security
RUN groupadd --system --gid 1000 rails && \
```